Sparse Approximation for Gaussian Process with Derivative Observations

Ang Yang, Cheng Li, Santu Rana, Sunil Gupta, and Svetha Venkatesh

Center for Pattern Recognition and Data Analytics,
Deakin University, Geelong, Australia
{leon.yang, cheng.l, santu.rana, sunil.gupta,
svetha.venkatesh}@deakin.edu.com

Abstract. We propose a sparse Gaussian process model to approximate full Gaussian process with derivatives when a large number of function observations t and derivative observations t' exist. By introducing a small number of inducing point m, the complexity of posterior computation can be reduced from $\mathcal{O}((t+t')^3)$ to $\mathcal{O}((t+t')m^2)$. We also find the usefulness of our approach in Bayesian optimisation. Experiments demonstrate the superiority of our approach.

Keywords: Sparse Gaussian process model, Bayesian optimisation, Derivative-based

1 Introduction

Gaussian process (GP) has been widely used for regression and classification tasks. It is a Bayesian method which specifies a prior distribution on the latent functions. Generic Gaussian process focuses on the observations of the function. Recent works have shown that the model accuracy can be improved by not only the function observations but also the derivative observations [1,2,3,4,5,6]. It is straightforward to use GP to model the combination of function observations and derivative observations due to the fact that the derivative of GP is still a GP [3].

A practical limitation of GP with derivative observations is that its computation expense increases rapidly with the size of training set. We shall recall that each derivative observation is a vector and each entry is a separate observation for GP. So the number of total observations also scales with the dimension. Generally, GP with derivatives requires $\mathcal{O}((t+t')^3)$ time to compute the Cholesky decomposition of a $(t+t') \times (t+t')$ covariance matrix, where t is the number of function observations and t' is the number of derivative observations. Therefore, it becomes very difficult to apply GP with derivative observations to large scale data sets. Similar limitation can also be found in the generic GP [7,8,9,10,11].

The focus of this paper is on deriving an efficient sparse GP model to approximate the full GP with derivative observations while preserving its predictive accuracy. We use a set of inducing variables, which contains the number

of m inducing points and $m \leq t$. Firstly, we assume that the function observations and derivative observations are conditionally independent by giving a set of inducing variables. Then the conditional distribution for a test point can be obtained by integrating out the inducing variables. To further decrease the costly computation, we introduce the fully independent conditional approximation where function observations, derivative observations and test points have no any deterministic relation on the inducing variables, so that we can ignore the covariance but remain the variance. This work is the first to develop a sparse GP model to approximate the full GP with derivative observations. The resultant sparse GP model is named sparse Gaussian process with derivatives, requiring the time $\mathcal{O}((t+t')m^2)$ for posterior computation. We have applied the proposed model in regression and Bayesian optimisation [12] on large scale datasets. The experimental results show the effectiveness of our proposed model.

2 Background

2.1 Gaussian process

Gaussian processes [13] is a strategy of specifying prior distributions over the space of smooth functions. The property of the Gaussian distribution allows us to compute the predictive mean and variance in the closed form. A GP is specified by its mean function $\mu(\boldsymbol{x})$ and covariance function $k(\boldsymbol{x}, \boldsymbol{x}')$. A sample from a Gaussian process is a function given as

$$f(\boldsymbol{x}) \sim \mathcal{N}(\mu(\boldsymbol{x}), k(\boldsymbol{x}, \boldsymbol{x}'))$$

where $\mathcal{N}$ is a Gaussian distribution and $\boldsymbol{x}$ denotes a D-dimensional vector. Without any loss in generality, the prior mean function can be assumed to be a zero function making the Gaussian process fully defined by the covariance function. A popular choice of kernel is the squared exponential function,

$$k(\boldsymbol{x}, \boldsymbol{x}') = \sigma_f^2 exp(-\tfrac{1}{2}\tfrac{\|\boldsymbol{x}-\boldsymbol{x}'\|^2}{\rho_l^2})$$

where ρ_l is the length scale, and σ_f is the amplitude.

Given a set of observations $\mathcal{D} = \{\boldsymbol{x}_i, \boldsymbol{f}_i\}_{i=1}^t$, the joint distribution of observations $\mathcal{D}$ and a new point $\{\boldsymbol{x}_{t+1}, f_{t+1}\}$ in GP is a multivariate Gaussian distribution

$$\begin{bmatrix} \boldsymbol{f}_{1:t} \\ f_{t+1} \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} K & \mathbf{k} \\ \mathbf{k}^T & k(\boldsymbol{x}_{t+1}, \boldsymbol{x}_{t+1}) \end{bmatrix}\right)$$

where $\boldsymbol{f}_{1:t} = \{f(\boldsymbol{x}_i)\}_{i=1}^t$, $\mathbf{k} = [\,k(\boldsymbol{x}_{t+1}, \boldsymbol{x}_1)\; k(\boldsymbol{x}_{t+1}, \boldsymbol{x}_2) \ldots k(\boldsymbol{x}_{t+1}, \boldsymbol{x}_t)\,]$, and K is the kernel matrix given by

$$K = \begin{bmatrix} k(\boldsymbol{x}_1, \boldsymbol{x}_1) & \ldots & k(\boldsymbol{x}_1, \boldsymbol{x}_t) \\ \vdots & \ddots & \vdots \\ k(\boldsymbol{x}_t, \boldsymbol{x}_1) & \ldots & k(\boldsymbol{x}_t, \boldsymbol{x}_t) \end{bmatrix}$$

Then the predictive distribution of f_{t+1} can be computed as

$$\mathcal{P}(f_{t+1}|\mathcal{D}_{1:t}, \boldsymbol{x}_{t+1}) = \mathcal{N}(\mu(\boldsymbol{x}_{t+1}), \sigma^2(\boldsymbol{x}_{t+1}))$$

with the predicted mean and variance

$$\mu(\boldsymbol{x}_{t+1}) = \mathbf{k}^T K^{-1} \boldsymbol{f}_{1:t}$$

$$\sigma^2(\boldsymbol{x}_{t+1}) = k(\boldsymbol{x}_{t+1}, \boldsymbol{x}_{t+1}) - \mathbf{k}^T K^{-1} \mathbf{k}.$$

If the observation is a noisy estimation of the actual function value

$$y = f(\boldsymbol{x}) + \xi$$

where $\xi \sim \mathcal{N}(0, \sigma^2_{noise})$, the predicted mean and variance can be computed as

$$\mu(\boldsymbol{x}_{t+1}) = \mathbf{k}^T [K + \sigma^2_{noise} I]^{-1} \boldsymbol{y}_{1:t}$$

$$\sigma^2(\boldsymbol{x}_{t+1}) = k(\boldsymbol{x}_{t+1}, \boldsymbol{x}_{t+1}) - \mathbf{k}^T [K + \sigma^2_{noise} I]^{-1} \mathbf{k}.$$

GP is prohibitive for large data sets because its training requires $\mathcal{O}(t^3)$ time due to the inversion of the covariance matrix, where t is the number of function observations.

2.2 Sparse Gaussian process

In sparse Gaussian process [9], it introduces a set of inducing observations $\mathcal{D}_u = \{\mathbf{M}_i, \mathbf{u}_i\}_{i=1}^m$ where $\mathbf{u} = \{\mathbf{u}_i\}_{i=1}^m$ contains values of the function at the points $\mathbf{M} = \{\mathbf{M}_i\}_{i=1}^m$, knowing as inducing points. Given prior distribution $p(\mathbf{u}) = \mathcal{N}(\mathbf{u} \mid \mathbf{0}, \boldsymbol{K}_{m,m})$, the training conditional distribution of $\boldsymbol{f}_{1:t}$ given $\mathbf{u}$ can be written as

$$p(\boldsymbol{f}_{1:t} \mid \mathbf{u}) = \mathcal{N}(\boldsymbol{f}_{1:t} \mid \boldsymbol{K}_{t,m} \boldsymbol{K}^{-1}_{m,m} \mathbf{u}, \boldsymbol{K}_{t,t} - \boldsymbol{Q}_{t,t}), \tag{1}$$

where $\boldsymbol{K}_{t,m}$ is the covariance matrix between all observations and inducing points and $\boldsymbol{K}_{m,m}$ is the covariance matrix between all inducing points. Besides, a shorthand notation $\boldsymbol{Q}_{a,b} = \boldsymbol{K}_{a,m} \boldsymbol{K}^{-1}_{m,m} \boldsymbol{K}_{m,b}$ is also introduced. Since $\mathbf{u}$ plays the role of observations so that the posterior mean is written as $\boldsymbol{K}_{t,m} \boldsymbol{K}^{-1}_{m,m} \mathbf{u}$. The covariance matrix has the form of the $\boldsymbol{K}_{t,t}$ minus a non-negative definite matrix $\boldsymbol{Q}_{t,t}$ which gives the measurement of how much information that $\mathbf{u}$ provides in $\boldsymbol{f}_{1:t}$. The test conditional $p(f_{t+1} \mid \mathbf{u})$ is formed the same way with Eq(1) as

$$p(f_{t+1} \mid \mathbf{u}) = \mathcal{N}(f_{t+1} \mid \boldsymbol{K}_{t+1,m} \boldsymbol{K}^{-1}_{m,m} \mathbf{u}, \boldsymbol{K}_{t+1,t+1} - \boldsymbol{Q}_{t+1,t+1}), \tag{2}$$

where f_{t+1} is a test point and $\boldsymbol{K}_{t+1,m}$ is the covariance matrix between the test point and inducing points.

To recover $p(\boldsymbol{f}_{1:t}, f_{t+1})$ we can simply integrating out $\mathbf{u}$ from the joint GP prior $p(\boldsymbol{f}_{1:t}, f_{t+1}, \mathbf{u})$ as

$$p(\boldsymbol{f}_{1:t},\, f_{t+1}) = \int\, p(\boldsymbol{f}_{1:t},\, f_{t+1},\, \mathbf{u})\, \mathrm{d}\mathbf{u} = \int p(\boldsymbol{f}_{1:t},\, f_{t+1} \mid \mathbf{u}) p(\mathbf{u})\, \mathrm{d}\mathbf{u}$$

In the first stage of approximation, we make the assumption that $\boldsymbol{f}_{1:t}$ and f_{t+1} are conditionally independent given $\mathbf{u}$, so the joint distribution of them can be written as

$$p(\boldsymbol{f}_{1:t},\, f_{t+1}) \approx q(\boldsymbol{f}_{1:t},\, f_{t+1}) = \int q(\boldsymbol{f}_{1:t} \mid \mathbf{u}) q(f_{t+1} \mid \mathbf{u}) p(\mathbf{u})\, \mathrm{d}\mathbf{u}$$

The sparse approximation is then derived by making additional assumptions about the $q(\boldsymbol{f}_{1:t} \mid \mathbf{u})$ and $q(f_{t+1} \mid \mathbf{u})$.

2.3 Gaussian process with derivative observations

The joint distribution of a function observation and a derivative observation is analytically tractable as the derivatives of a Gaussian process is still a GP [3]. The joint distribution is given as

$$\begin{bmatrix} f \\ \nabla f \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0},\, \begin{bmatrix} k_{[f,f]} & k_{[f,\nabla f]} \\ k_{[\nabla f,f]} & k_{[\nabla f,\nabla f]} \end{bmatrix}\right) \tag{3}$$

In terms of the squared exponential covariance function, the covariance between function values and partial derivatives can be written as [1]

$cov(f^i, \frac{\partial f^j}{\partial x_g^{(j)}}) = \sigma_f^2 exp(-\frac{1}{2}\sum_{b=1}^{D} \rho_l^{-2}(x_b^{(i)} - x_b^{(j)})^2) \times (\rho_l^{-2}(x_g^{(i)} - x_g^{(j)}))$

and covariance between partial derivatives is given as

$cov(\frac{\partial f^j}{\partial x_g^{(i)}}, \frac{\partial f^j}{\partial x_h^{(j)}}) =$
$\sigma_f^2 exp(-\frac{1}{2}\sum_{b=1}^{D} \rho_l^{-2}(x_b^{(i)} - x_b^{(j)})^2) \times \rho_l^{-2}(\delta_{gh} - \rho_l^{-2}(x_h^{(i)} - x_h^{(j)})(x_g^{(i)} - x_g^{(j)}))$

where $\delta_{gh} = 1$ if $g = h$, and $\delta_{gh} = 0$ if $g \neq h$.

Suppose we have extra derivative observations upon giving function observations $\mathcal{D}$ and we denote the set as $\mathcal{D}' = \{\boldsymbol{x}_j,\, \nabla \boldsymbol{f}_j\}_{j=1}^{t'}$, where$\nabla \boldsymbol{f}_{1,t'} = \{\nabla f(\boldsymbol{x}_j)\}_{j=1}^{t'}$. Now using GP we can derive the posterior over a new function value f_{t+1} at $\boldsymbol{x}_{t+1}$ when given a set of observations of the function values and a set of derivative information. We use $\bar{K}_{[\boldsymbol{f}_{1:t}, \nabla \boldsymbol{f}_{1:t'}]}$ to denote the joint covariance matrix over a set of observations of function values and the derivatives. Then the new joint distribution for $[\boldsymbol{f}_{1:t}, \nabla \boldsymbol{f}_{1:t'},\, f_{t+1}]$ is showing as

$$\begin{bmatrix} \boldsymbol{f}_{1:t} \\ \nabla \boldsymbol{f}_{1:t'} \\ f_{t+1} \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0},\, \begin{bmatrix} \bar{K}_{[\boldsymbol{f}_{1:t}, \nabla \boldsymbol{f}_{1:t'}]} & \bar{\mathbf{k}} \\ \mathbf{k}^T & k(\boldsymbol{x}_{t+1},\, \boldsymbol{x}_{t+1}) \end{bmatrix}\right) \tag{4}$$

3 Proposed Framework

Full GP with derivative observations becomes prohibitive when a large number of function observations and derivative observations exist due to the high computational cost. We propose a sparse model to approximate the full GP with derivative observations while preserving the desirable properties of it. Firstly, we derive our model named sparse Gaussian process with derivatives (SGPD). Then we apply the proposed sparse method as the probabilistic model in Bayesian optimisation.

3.1 Sparse Gaussian Process with Derivatives for Regression

Similar with sparse GP, we induce a set of variables $\mathbf{u}$ and use the same notations as before. We can easily get the following equation by marginalizing out $\mathbf{u}$

$$p(\boldsymbol{f}_{1:t}, \nabla \boldsymbol{f}_{1:t'}, f_{t+1}) = \int p(\boldsymbol{f}_{1:t}, \nabla \boldsymbol{f}_{1:t'}, f_{t+1} \mid \mathbf{u}) p(\mathbf{u}) \mathrm{d}\mathbf{u} \tag{5}$$

Further we make the assumption that function observation, derivative observations and test points only depend on the inducing variables $\mathbf{u}$. Then we can rewrite the Eq(5) as

$$p(\boldsymbol{f}_{1:t}, \nabla \boldsymbol{f}_{1:t'}, f_{t+1}) = \int p(\boldsymbol{f}_{1:t} \mid \mathbf{u}) p(\nabla \boldsymbol{f}_{1:t'} \mid \mathbf{u}) p(f_{t+1} \mid \mathbf{u}) p(\mathbf{u}) \mathrm{d}\mathbf{u} \tag{6}$$

Therefore we next show how to compute the conditional distributions $p(\boldsymbol{f}_{1:t} \mid \mathbf{u})$, $p(\nabla \boldsymbol{f}_{1:t'} \mid \mathbf{u})$ and $p(f_{t+1} \mid \mathbf{u})$ using both exact expressions and approximate expressions.

Given the prior $p(\mathbf{u}) = \mathcal{N}(\mathbf{u} \mid \mathbf{0}, \boldsymbol{K}_{m,m})$, the exact expression for $p(\boldsymbol{f}_{1:t} \mid \mathbf{u})$ can be computed as

$$p(\boldsymbol{f}_{1:t} \mid \mathbf{u}) = \mathcal{N}(\boldsymbol{f}_{1:t} \mid \boldsymbol{K}_{t,m} \boldsymbol{K}_{m,m}^{-1} \mathbf{u}, \boldsymbol{K}_{t,t} - \boldsymbol{Q}_{t,t}) \tag{7}$$

where we recall $\boldsymbol{Q}_{a,b} = \boldsymbol{K}_{a,m} \boldsymbol{K}_{m,m}^{-1} \boldsymbol{K}_{m,b}$. Similarly, the conditional distribution for a test point $\{\boldsymbol{x}_{t+1}, f_{t+1}\}$ is

$$p(f_{t+1} \mid \mathbf{u}) = \mathcal{N}(f_{t+1} \mid \boldsymbol{K}_{t+1,m} \boldsymbol{K}_{m,m}^{-1} \mathbf{u}, \boldsymbol{K}_{t+1,t+1} - \boldsymbol{Q}_{t+1,t+1}) \tag{8}$$

and the conditional distribution for derivative observations is

$$p(\nabla \boldsymbol{f}_{1:t'} \mid \mathbf{u}) = \mathcal{N}(\nabla \boldsymbol{f}_{1:t'} \mid \boldsymbol{K}_{t',m} \boldsymbol{K}_{m,m}^{-1} \mathbf{u}, \boldsymbol{K}_{t',t'} - \boldsymbol{Q}_{t',t'}) \tag{9}$$

It is noted that $\boldsymbol{K}_{t',m}$ is the covariance matrix between derivative observations $\nabla \boldsymbol{f}_{1:t'}$ and inducing variables $\mathbf{u}$ and $\boldsymbol{Q}_{t',t'} = \boldsymbol{K}_{t',m} \boldsymbol{K}_{m,m}^{-1} \boldsymbol{K}_{m,t'}$.

There are several methods to approximate conditional distributions in Eq(7, 8, 9) and Joaquin and Carl [9] have provided a unifying view of them. In our framework, we use the fully independent conditional (FIC) approximation where training function observations, derivative observations and test points are fully independent on $\mathbf{u}$.

Now we can derive the approximate expression of $p(\boldsymbol{f}_{1:t} \mid \mathbf{u})$ as

$$q(\boldsymbol{f}_{1:t} \mid \mathbf{u}) = \mathcal{N}(\boldsymbol{K}_{t,m}\boldsymbol{K}_{m,m}^{-1}\mathbf{u},\, diag[\boldsymbol{K}_{t,t} - \boldsymbol{Q}_{t,t}]) \tag{10}$$

where $diag[\boldsymbol{K}_{t,t} - \boldsymbol{Q}_{t,t}]$ is a diagonal matrix whose elements match the diagonal of $\boldsymbol{K}_{t,t} - \boldsymbol{Q}_{t,t}$ so that we only keep the variance information of function observations themselves and ignore covariance between function observations and inducing points. Then $\boldsymbol{f}_{1:t}$ have no any deterministic relation on $\mathbf{u}$. Likewise, the approximate expression for $p(f_{t+1} \mid \mathbf{u})$ can write as

$$q(f_{t+1} \mid \mathbf{u}) = \mathcal{N}(\boldsymbol{K}_{t+1,m}\boldsymbol{K}_{m,m}^{-1}\mathbf{u},\, diag[\boldsymbol{K}_{t+1,t+1} - \boldsymbol{Q}_{t+1,t+1}]) \tag{11}$$

and the approximated format for $p(\nabla \boldsymbol{f}_{1:t'} \mid \mathbf{u})$ is

$$q(\nabla \boldsymbol{f}_{1:t'} \mid \mathbf{u}) = \mathcal{N}(\boldsymbol{K}_{t',m}\boldsymbol{K}_{m,m}^{-1}\mathbf{u},\, diag[\boldsymbol{K}_{t',t'} - \boldsymbol{Q}_{t',t'}]) \tag{12}$$

By substituting Eq(10, 11, 12) into Eq(6) we can give the FIC approximate joint distribution

$$q(\boldsymbol{f}_{1:t},\, \nabla \boldsymbol{f}_{1:t'},\, f_{t+1}) = \mathcal{N}\left(\mathbf{0},\, \begin{bmatrix} \boldsymbol{K}\boldsymbol{Q}_t & \boldsymbol{Q}_{t,t'} & \boldsymbol{Q}_{t,t+1} \\ \boldsymbol{Q}_{t',t} & \boldsymbol{K}\boldsymbol{Q}_{t'} & \boldsymbol{Q}_{t',t+1} \\ \boldsymbol{Q}_{t+1,t} & \boldsymbol{Q}_{t+1,t'} & \boldsymbol{K}\boldsymbol{Q}_{t+1} \end{bmatrix}\right) \tag{13}$$

where $\boldsymbol{K}\boldsymbol{Q}_t = \boldsymbol{Q}_{t,t} + diag[\boldsymbol{K}_{t,t} - \boldsymbol{Q}_{t,t}]$, $\boldsymbol{K}\boldsymbol{Q}_{t'} = \boldsymbol{Q}_{t',t'} + diag[\boldsymbol{K}_{t',t'} - \boldsymbol{Q}_{t',t'}]$ and $\boldsymbol{K}\boldsymbol{Q}_{t+1} = \boldsymbol{Q}_{t+1,t+1} + diag[\boldsymbol{K}_{t+1,t+1} - \boldsymbol{Q}_{t+1,t+1}]$.
The posterior distribution of the test point $\{\boldsymbol{x}_{t+1},\, f_{t+1}\}$ is a Gaussian distribution

$$f(\boldsymbol{x}_{t+1}) \sim \mathcal{N}(\tilde{\mu}(\boldsymbol{x}_{t+1}), \tilde{\sigma}^2(\boldsymbol{x}_{t+1}))$$

with mean and variance as

$$\tilde{\mu}(\boldsymbol{x}_{t+1}) = \boldsymbol{Q}_{t+1,tt'}\boldsymbol{K}_{t,t'}^{FIC\,-1}[\boldsymbol{f}_{1:t}, \nabla \boldsymbol{f}_{1:t'}]$$

$$\tilde{\sigma}^2(\boldsymbol{x}_{t+1}) = \boldsymbol{K}_{t+1,t+1} - \boldsymbol{Q}_{t+1,tt'}\boldsymbol{K}_{t,t'}^{FIC\,-1}\boldsymbol{Q}_{tt',t+1}$$

where $\boldsymbol{K}_{t,t'}^{FIC} = \begin{bmatrix} \boldsymbol{Q}_{t,t} + diag[\boldsymbol{K}_{t,t} - \boldsymbol{Q}_{t,t}] & \boldsymbol{Q}_{t,t'} \\ \boldsymbol{Q}_{t',t} & \boldsymbol{Q}_{t',t'} + diag[\boldsymbol{K}_{t',t'} - \boldsymbol{Q}_{t',t'}] \end{bmatrix}$,

$\boldsymbol{Q}_{t+1,tt'} = [\boldsymbol{Q}_{t+1,t},\, \boldsymbol{Q}_{t+1,t'}]$ which combines $\boldsymbol{Q}_{t+1,t}$ and $\boldsymbol{Q}_{t+1,t'}$ into one matrix and $\boldsymbol{Q}_{tt',t+1} = \boldsymbol{Q}_{t+1,tt'}^T$.

If the function observation is a noisy estimation of the actual function value, the predicted mean and variance can be computed as

$$\tilde{\mu}(\boldsymbol{x}_{t+1}) = \boldsymbol{Q}_{t+1,tt'}[\boldsymbol{K}_{t,t'}^{FIC} + \begin{bmatrix} \sigma_{noise}^2 \\ 0 \end{bmatrix} I]^{-1}[\boldsymbol{y}_{1:t}, \nabla \boldsymbol{f}_{1:t'}]$$

$$\tilde{\sigma}^2(\boldsymbol{x}_{t+1}) = \boldsymbol{K}_{t+1,t+1} - \boldsymbol{Q}_{t+1,tt'}[\boldsymbol{K}_{t,t'}^{FIC} + \begin{bmatrix} \sigma_{noise}^2 \\ 0 \end{bmatrix} I]^{-1}\boldsymbol{Q}_{tt',t+1}$$

Complexity analysis In full GP with derivatives the computational complexity for training is $\mathcal{O}((t+t')^3)$ where t is the number of function observations and t' is the number of the derivative observations. The proposed method which introduced a set of inducing points $\mathbf{u}$, bring the complexity down to $\mathcal{O}((t+t')m^2)$, where m is the number of inducing points and $m \leq t$.

3.2 Application to Bayesian optimisation

Bayesian optimisation (BO) has two main components [14]. The one is to model the latent function using Gaussian process as a prior. The other component is to search the next point where to perform the experiment. The search for the next point is guided by a surrogate function, called acquisition function. Acquisition functions are designed to trade off exploitation and exploration. High exploitation means the areas where the mean prediction for function values are high. High exploration means the areas where the epistemic uncertainty about the function values is high. There are different types of acquisition functions. Popular ones include EI [15], PI [16] and UCB [17]. In this paper, we use EI as the criteria.

Bayesian optimisation using Gaussian process as a prior can be extremely costly when there is a large number of function observations and derivative observations and it is because the size of the covariance matrix is too large. Therefore, we replace the modeling part with our method SGPD so that the time taken during optimisation will not increase regardless of the increasing number of function and derivative observations. The proposed algorithm is showing in Algorithm 1.

Algorithm 1 Bayesian Optimisation Using SGPD

Input data: $\mathcal{D} = \{\boldsymbol{x}_i, \boldsymbol{f}_i\}_{i=1}^{t}$, $\mathcal{D}' = \{\boldsymbol{x}_j, \nabla \boldsymbol{f}_j\}_{j=1}^{t'}$ and $\mathcal{D}_u = \{\mathbf{M}_i, \mathbf{u}_i\}_{i=1}^{m}$
1: **for** $n = 1, 2, \ldots$ **do**
2: Model SGPD using input data.
2: Find $\boldsymbol{x}_{t+1}$ by optimizing the acquisition function $EI(\boldsymbol{x}|\mathcal{D})$:
$\boldsymbol{x}_{t+1} = argmax_{\boldsymbol{x}} EI(\boldsymbol{x}|\mathcal{D})$
3: Evaluate the objective function: $y_{t+1} = f(\boldsymbol{x}_{t+1}) + \xi$
4: Augment the observation set $\mathcal{D} = \mathcal{D} \cup (\boldsymbol{x}_{t+1}, y_{t+1})$,
$\mathcal{D}' = \mathcal{D}' \cup \{\boldsymbol{x}_{1:t+1'}, \boldsymbol{f}_{1:t+1'}\}$
5: **end for**

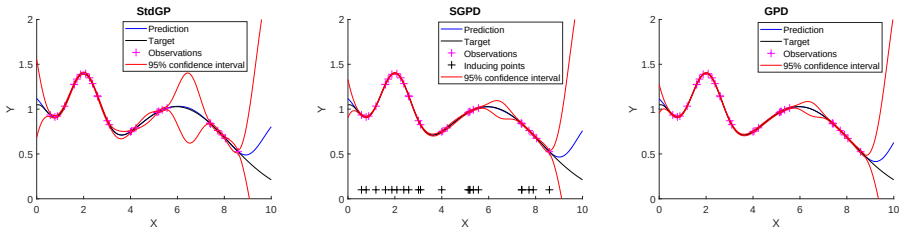
4 Experiments

We evaluate our method on regression tasks and application of Bayesian optimisation. We compare the proposed method SGPD with the following baselines for regression:

– Standard Gaussian process (StdGP)
– Sparse Gaussian process (SGP)
– Gaussian process with derivatives (GPD)

In Bayesian optimisation tasks, we only compare with StdGP and GPD. For all GPs we use the SE kernel with the hyperparameters - the isotropic length scale $\rho_l = 0.8$, signal variance $\sigma_f^2 = 1$ and noise variance $\sigma_{noise}^2 = (0.01)^2$. In terms of the inducing variables selection, we use a randomly selected subset of function observations as inducing points in all of our experiments.

We visualise the results for 1D function in Figure 1. We use 30 function observations ($t = 30$) and their derivatives ($t' = 30$) as training data and use 450 points for testing. We randomly select 70% of t (21 points) as inducing points (Derivative observations will not be used as inducing points in all of our cases), then compare our method with StdGP and GPD. The mean functions of 3 methods look similar but the variance of our method SGPD has a very close format with GPD.



(a) (b) (c)

Fig. 1: The visualisation of 3 methods for regression on 1D function (a) Standard Gaussian Process (b) Sparse Gaussian Process with Derivatives and (c) Gaussian Process with Derivatives.

4.1 Regression on benchmark functions

In regression tasks, we run each algorithm 50 trials with different initialisation and report the mean square error (MSE). Within each trial of SGP and SGPD, we run 50 times on randomly selecting subset to calculate the average MSE of each trial. We test our algorithm on five benchmark functions below:

1. Function with multiple local optima-1D (1D).
2. Branin function-2D (2D).
3. Hartmann-3D (3D).
4. Hartmann-4D (4D).
5. Hartmann-6D (6D).

For different functions, we use the number of training and test data referring to Table 1. The number of function observations t shows in column 2 while column

3 indicates the number of derivative observations t' . We use derivative observations at each dimension so that t' is equal to $D * t$, where D is the dimensional of the input space. We set the number of inducing points m as 70% of function observations (t) for 1D and 2D functions while setting 50% for 3D, 4D and 6D functions. We also set up an experiment to discover how our method performs if we use all function observations as inducing points ($m = t$). We summarise the MSE of each algorithm in Table 2. For all experiments, GPD has shown the best result since function observations and derivatives observations are fully used. Our method SGPD closely approximates the GPD and shows a better result than SGP as well as StdGP in all regression tasks. Besides, the setting of $m = t$ with our method achieves very comparable performance to GPD while requiring less computational complexity. While requires less computational complexity, the setting of $m = t$ in SGPD achieves very comparable performance to GPD. For example, GPD requires 300 function observations plus 1800 derivative observations in 6D case, but SGPD ($m = t$) only incorporates 300 function observations.

Table 1. Functions for Regression

	Function observations	Derivative observations	Test points
1D	30	30	450
2D	200	400	800
3D	200	600	800
4D	300	1200	900
6D	300	1800	900

Table 2. Comparison of MSE results among each method for regression tasks.

	StdGP	SGP	GPD
1D	4.81e-5±1.23e-4	6.20e-5±1.51e-4 (0.7t)	4.54e-6±1.19e-5
2D	0.0164±0.0200	0.0397±0.0408 (0.7t)	6.80e-4±0.0014
3D	0.0786±0.0145	0.0787±0.0145 (0.5t)	0.0319±0.0051
4D	0.0453±0.0064	0.0473±0.0066 (0.5t))	0.0106±0.0024
6D	0.0176±0.0038	0.0177±0.0026 (0.5t))	0.0053±0.0016

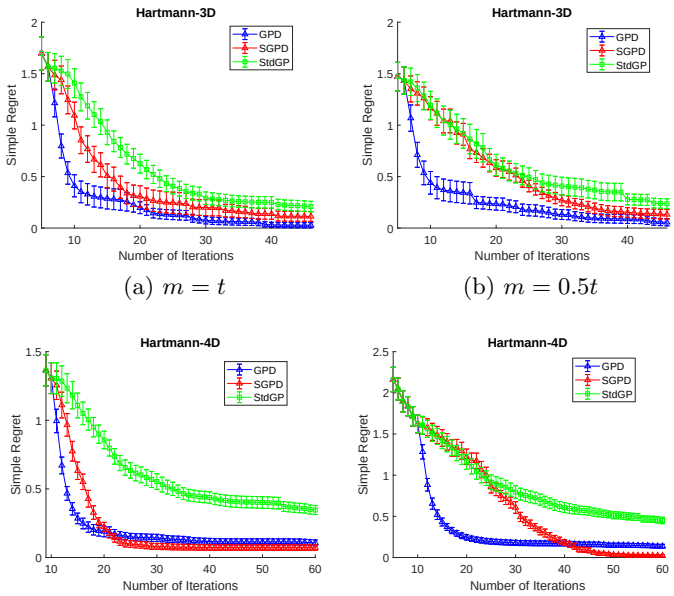
	SGPD	SGPD ($m = t$)
1D	4.68e-5±2.12e-4 (0.7t)	8.00e-6±2.92e-5
2D	0.0114±0.0145 (0.7t)	0.0014±0.0022
3D	0.0368±0.0072 (0.5t)	0.0338±0.0052
4D	0.0401±0.0079 (0.5t)	0.0131±0.0030
6D	0.0162±0.0019 (0.5t)	0.0092±0.0015

4.2 Experiments with Bayesian optimisation

We apply our method on Bayesian optimisation for two benchmark functions:

1. Hartmann-3D, where $\boldsymbol{x}^* = (0.1146, 0.5556, 0.8525)$ is the global minimum location with function value of $f(\boldsymbol{x}^*) = -3.8628$.
2. Hartmann-4D, where $\boldsymbol{x}^* = (0.1873, 0.1906, 0.5566, 0.2647)$ is the global minimum location with function value of $f(\boldsymbol{x}^*) = -3.1355$.

We use EI as acquisition function and DIRECT [18] as optimiser to optimise EI. We run each algorithm 30 trials with different initialisation and report the simple regret and standard errors at the end. Simple regret is defined as $r = f(\boldsymbol{x}^*) - f(\boldsymbol{x}^+)$ where $f(\boldsymbol{x}^*)$ is the global optimum and $f(\boldsymbol{x}^+) = max_{\boldsymbol{x} \in \{\boldsymbol{x}_{1:t}\}} f(\boldsymbol{x})$ which is the current best value. Figure 2 plots the simple regret vs iteration for all experiments. BO with GPD performs the best in all three algorithms. It is easy to explain since all function observations and derivative observations have been incorporated in the algorithm. In Figure 2a, BO with our method SGPD using 100% function observations as inducing points outperforms BO with StdGP along the entire process. Figure 2b demonstrates the result of using 50% function observation as inducing points. Although our method performs similar to StdGP at the beginning, but jump ahead after 23 iterations as more information come into SGPD. But we should emphasize that the computational cost of SGPD is less than both GPD and StdGP in this case. We also receive similar results for Hartmann-4D, and illustrate in Figure 2c and Figure 2d respectively.



(a) $m = t$ (b) $m = 0.5t$

(c) $m = t$ (d) $m = 0.5t$

Fig. 2: Simple regret vs iterations for (a) Hartmann-3D using 100% t as inducing points (b)Hartmann-3D using 50% t as inducing points (c) Hartmann-4D using 100% t as inducing points(d) Hartmann-4D using 50% t as inducing points.

5 Conclusion

We propose a novel method for speeding up Gaussian process with derivatives in the case of a large number of function observations and derivatives observations. We also investigate the usability of our method in large scale Bayesian optimisation. For all experiments, our proposed approach closely approximate full Gaussian process with derivatives. In future, we consider to automatically estimate locations and number of inducing points and extend to other applications.

Acknowledgment This research was partially funded by the Australian Government through the Australian Research Council (ARC). Prof Venkatesh is the recipient of an ARC Australian Laureate Fellowship (FL170100006).

References

1. Riihimäki, J., Vehtari, A.: Gaussian processes with monotonicity information. In: Proceedings of the Thirteenth International Conference on AIStat. (2010)
2. Siivola, E., Vehtari, A., Vanhatalo, J., González, J.: Bayesian optimization with virtual derivative sign observations. arXiv preprint arXiv:1704.00963 (2017)
3. Solak, E., Murray-Smith, R., Leithead, W.E., Leith, D.J., Rasmussen, C.E.: Derivative observations in gaussian process models of dynamic systems. In: NIPS. (2003)
4. Jauch, M., Peña, V.: Bayesian optimization with shape constraints. arXiv preprint arXiv:1612.08915 (2016)
5. Li, C., Rana, S., Gupta, S., Nguyen, V., Venkatesh, S.: Bayesian optimization with monotonicity information. (2017)
6. Yang, A., Li, C., Rana, S., Gupta, S., Venkatesh, S.: Efficient bayesian optimisation using derivative meta-model. In: The 15th Pacific Rim International Conference on Artificial Intelligence, 2018. (2018)
7. Snelson, E., Ghahramani, Z.: Sparse gaussian processes using pseudo-inputs. In: Advances in neural information processing systems. (2006) 1257–1264
8. Snelson, E., Ghahramani, Z.: Variable noise and dimensionality reduction for sparse gaussian processes. arXiv preprint arXiv:1206.6873 (2012)
9. Quiñonero-Candela, J., Rasmussen, C.E.: A unifying view of sparse approximate gaussian process regression. Journal of Machine Learning Research **6**(Dec) (2005) 1939–1959
10. Li, C., Gupta, S., Rana, S., Nguyen, V., Venkatesh, S., Shilton, A.: High dimensional bayesian optimization using dropout. arXiv preprint arXiv:1802.05400 (2018)
11. Rana, S., Li, C., Gupta, S., Nguyen, V., Venkatesh, S.: High dimensional bayesian optimization with elastic gaussian process. In: International Conference on Machine Learning. (2017) 2883–2891
12. Jones, D.R., Schonlau, M., Welch, W.J.: Efficient global optimization of expensive black-box functions. Journal of Global optimization **13**(4) (1998) 455–492
13. Rasmussen, C.E., Williams, C.K.: Gaussian processes for machine learning. Volume 1. MIT press Cambridge (2006)
14. Brochu, E., Cora, V.M., De Freitas, N.: A tutorial on bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. arXiv preprint arXiv:1012.2599 (2010)

15. Mockus, J.: Application of bayesian approach to numerical methods of global and stochastic optimization. Journal of Global Optimization **4**(4) (1994) 347–365
16. Kushner, H.J.: A new method of locating the maximum point of an arbitrary multipeak curve in the presence of noise. Journal of BE (1964)
17. Srinivas, N., Krause, A., Kakade, S.M., Seeger, M.: Gaussian process optimization in the bandit setting: No regret and experimental design. arXiv preprint arXiv:0912.3995 (2009)
18. Finkel, D.E.: Direct optimization algorithm user guide. CRSC (2003)